Naive Bayes

A Practical Guide to Probabilistic Classification

GitHub-ready Machine Learning Notes

Naive Bayes is a supervised machine learning algorithm mainly used for classification. It is based on Bayes' Theorem and assumes that features are conditionally independent given the class.

1. What is Naive Bayes?

Naive Bayes predicts the class of an observation by calculating the probability of each possible class given the observed features. The class with the highest posterior probability becomes the prediction.

Common applications: Spam detection, sentiment analysis, news classification, document classification, customer categorization, and medical classification.

2. Why is it called 'Naive'?

The algorithm makes a simplifying assumption that features are conditionally independent given the class. In real-world data, features are often related, so this assumption is considered naive.

3. Bayes' Theorem

```
P(A|B) = [P(B|A) × P(A)] / P(B)
```

Posterior: probability of A after observing B. **Likelihood:** probability of B given A. **Prior:** probability of A before observing B. **Evidence:** probability of B.

4. Naive Bayes Formula

```
P(C|X) ∝ P(C) × ∏ P(x■|C)
```

The model calculates the posterior probability for every possible class and predicts the class with the highest probability.

5. Simple Example — Spam Detection

Suppose an email contains the words FREE, OFFER, and MONEY. Naive Bayes calculates the probability that the email is Spam and Not Spam.

```
P(Spam | FREE, OFFER, MONEY)     = 0.92
P(Not Spam | FREE, OFFER, MONEY) = 0.08

Prediction = Spam
```

6. How Naive Bayes Works

```
Training Data
    ↓
Calculate class probabilities
    ↓
Calculate feature probabilities
    ↓
Apply Bayes' Theorem
    ↓
```

```
Calculate posterior probabilities
    ↓
Choose highest probability
    ↓
Final Prediction
```

7. Types of Naive Bayes

Type	Best Used For	Scikit-learn
Gaussian NB	Continuous numerical features	GaussianNB
Multinomial NB	Counts, frequencies, text	MultinomialNB
Bernoulli NB	Binary features	BernoulliNB

8. Gaussian Naive Bayes

Gaussian Naive Bayes is suitable for continuous numerical features when their distributions within classes can reasonably be modeled using a Gaussian distribution.

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

9. Multinomial Naive Bayes

Multinomial Naive Bayes is commonly used for text classification with count-based or TF-IDF features.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(text_train)
X_test_tfidf = vectorizer.transform(text_test)

model = MultinomialNB()
model.fit(X_train_tfidf, y_train)
y_pred = model.predict(X_test_tfidf)
```

10. Bernoulli Naive Bayes

Bernoulli Naive Bayes is useful when features are binary, such as whether a word is present or absent.

```
from sklearn.naive_bayes import BernoulliNB

model = BernoulliNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

11. Laplace Smoothing

If a feature has never appeared in a particular class, its probability may become zero. Laplace smoothing prevents this zero-frequency problem.

```
P(x■|C) = [count(x■,C) + α] / [count(C) + αn]

model = MultinomialNB(alpha=1.0)
```

12. Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

13. Complete Gaussian Naive Bayes Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

14. Predicted Probabilities

Naive Bayes can provide probability estimates for each class using predict_proba().

```
y_prob = model.predict_proba(X_test)
print(y_prob)

# Binary classification: probability of positive class
y_prob = model.predict_proba(X_test)[:, 1]
```

15. Evaluation Metrics

Accuracy: proportion of predictions that are correct.
Precision: how many predicted positives are actually positive.
Recall: how many actual positives were identified.
F1-score: harmonic mean of precision and recall.
Confusion Matrix: summarizes correct and incorrect predictions by class.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

16. Advantages

• Fast and computationally efficient
• Simple to implement
• Works well with high-dimensional data
• Often effective with small datasets
• Supports multiclass classification
• Can provide class probabilities
• Excellent baseline for many text-classification problems

17. Limitations

• Conditional independence assumption may be unrealistic
• Correlated features can affect performance
• Zero-frequency problems require smoothing
• Different variants suit different data types
• Probabilities may not always be perfectly calibrated

18. Naive Bayes vs Logistic Regression

Feature	Naive Bayes	Logistic Regression
Core idea	Bayes' theorem	Learn coefficients
Type	Probabilistic classifier	Linear classifier
Feature assumption	Conditional independence	No independence assumption
Training	Usually very fast	Fast
Text classification	Excellent baseline	Often strong
Probability output	Yes	Yes

Nonlinear patterns	Limited	Limited without feature engineering

19. Naive Bayes vs KNN

Feature	Naive Bayes	KNN
Learning type	Probabilistic	Instance-based
Prediction	Probability calculation	Neighbor search
Scaling	Depends on variant	Usually important
Training	Very fast	Minimal
Prediction	Usually fast	Can be expensive
Text data	Often excellent	Usually not first choice
High-dimensional data	Often effective	Can suffer from dimensionality

20. When Should You Use Naive Bayes?

Use Naive Bayes when you have a classification problem, need a fast baseline, have many features, work with text, or need a simple probabilistic model.

21. Real-World Applications

Spam Detection: Spam vs Not Spam.
Sentiment Analysis: Positive, Negative, or Neutral.
News Classification: Sports, Technology, Business, etc.
Document Classification: Automatically categorize documents.

22. Practical ML Workflow

```
Define Problem
    ↓
Collect Data
    ↓
EDA + Data Cleaning
    ↓
Feature Engineering / Vectorization
    ↓
Train-Test Split
    ↓
Choose NB Variant
    ↓
Train Model
    ↓
Predict
    ↓
Evaluate
    ↓
Tune + Document
```

23. Common Mistakes

• Choosing the wrong Naive Bayes variant
• Data leakage during preprocessing
• Evaluating only on training data
• Looking only at accuracy on imbalanced data
• Ignoring correlated features
• Forgetting smoothing where appropriate

24. Key Takeaways

```
Prior Probability + Feature Evidence
              ↓
       Bayes' Theorem
              ↓
   Posterior Probabilities
              ↓
    Highest Probability
              ↓
         Prediction
```

Naive Bayes is a fast, simple, and useful probabilistic classifier. The most important concepts are Bayes' theorem, prior probability, likelihood, posterior probability, conditional independence, Gaussian/Multinomial/Bernoulli variants, Laplace smoothing, and evaluation metrics.

25. Suggested GitHub Project Structure

```
naive-bayes/
■■■ data/
■   ■■■ dataset.csv
■■■ notebooks/
■   ■■■ naive_bayes.ipynb
■■■ images/
■   ■■■ confusion_matrix.png
■   ■■■ model_performance.png
■■■ README.md
■■■ requirements.txt
■■■ Naive_Bayes_Guide.pdf
```

26. Portfolio Project Idea — Spam Detection

```
Email Dataset
    ↓
Text Cleaning
    ↓
Train-Test Split
    ↓
TF-IDF Vectorization
    ↓
Multinomial Naive Bayes
    ↓
Predictions
    ↓
Confusion Matrix + F1
    ↓
Error Analysis
    ↓
Conclusion
```

A Spam Detection project demonstrates a complete ML pipeline from raw text to preprocessing, feature extraction, model training, evaluation, and interpretation.